

UAM Azcapotzalco, División de CBI

Club de Algoritmos

UAM Azcapotzalco

Entrenador: Dr. Rodrigo Alexander Castro Campos

hash.sh	4 líneas
# Hashea un archivo, ignorando todos los espacios en blanco y # comentarios. Úsalo para verificar que el código fue escrito # correctamente. cpp -dD -P -fpreprocessed tr -d '[:space:]' md5sum cut -c-6	
.bashrc	2 líneas
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++20 \ -fsanitize=undefined,address'	

Aritmética y primalidad (2)

Aritmética modular

Descripción: Operadores para aritmética modular. Supone que <i>MOD</i> es primo.	
Requiere: Inverso modular	79e848, 31 líneas

```
template<int MOD>
struct modular_int {
    int v;

    modular_int(int64_t u = 0)
    : v(u % MOD) {
        v += (v < 0) * MOD;
    }
    modular_int& operator+=(modular_int o) {
        if ((v += o.v) >= MOD) v -= MOD;
        return *this;
    }
    modular_int& operator-=(modular_int o) {
        if ((v -= o.v) < 0) v += MOD;
        return *this;
    }
    modular_int& operator*=(modular_int o) {
        v = (int64_t)v * o.v % MOD;
        return *this;
    }
    modular_int& operator/=(modular_int o) {
        return (*this) *= modular_int(inverso(o.v, MOD));
    }
    auto operator<=>(const modular_int& o) const = default;
    explicit operator int( ) const { return v; }
    friend modular_int operator+(modular_int a, modular_int b) {
        ↪ return a += b; }
    friend modular_int operator-(modular_int a, modular_int b) {
        ↪ return a -= b; }
    friend modular_int operator*(modular_int a, modular_int b) {
        ↪ return a *= b; }
    friend modular_int operator/(modular_int a, modular_int b) {
        ↪ return a /= b; }
};
//using gf = modular_int<1e9+7>;
```

Criba de Eratóstenes

Descripción: Criba de Eratóstenes para encontrar todos los números primos hasta un límite dado. Los números primos se almacenan en <i>primos</i> , y <i>factor</i> guarda el guarda el mayor factor primo de cada número.	
Complejidad: $\mathcal{O}(n \log \log n)$	9ad675, 21 líneas

```
struct criba {
    int tope;
    vector<bool> es_primo;
    vector<int> menor_factor;

    criba(int t)
    : tope(t), es_primo(t + 1, true), menor_factor(t + 1) {
        es_primo[0] = es_primo[1] = false;
        for (int i = 2; i <= tope; i++) {
            if (es_primo[i]) {
                menor_factor[i] = i;
                for (int j = i * i; j <= tope; j += i) {
                    es_primo[j] = false;
                    if (menor_factor == 0) {
                        menor_factor[j] = i;
                    }
                }
            }
        }
    }
};
```

Factorización básica

Descripción: Devuelve un vector ordenado con los factores primos de <i>n</i> .	
Complejidad: $\mathcal{O}(\sqrt{n})$	6631b4, 14 líneas

```
vector<int64_t> factoriza(int64_t n) {
    vector<int64_t> factores;
    int64_t raíz = round(sqrt(n));
    for (int64_t x = 2; x <= raíz; ++x) {
        while (n % x == 0) {
            factores.push_back(x);
            n /= x;
        }
    }
    if (n > 1) {
        factores.push_back(n);
    }
    return factores;
}
```

Factorización de números grandes

Descripción: Algoritmo de Pollard Rho. <i>factoriza</i> toma un número entero $n \geq 2$ y devuelve un vector con los factores primos de <i>n</i> en orden arbitrario. Por ejemplo, para <i>n</i> = 2299, puede devolver {11, 19, 11}.	
Complejidad: $\mathcal{O}\left(n^{1/4}\right)$	
Requiere: Primalidad Miller Rabin	3ec062, 35 líneas

```
int64_t factor_pollard_rho(int64_t n, int64_t c) {
    int64_t x = 2, y = 2, k = 2;
    for (int64_t i = 2; ; ++i) {
        x = ((__int128_t)x * x) % n + c;
        if (x >= n) { x -= n; }
        int64_t d = gcd(x - y, n);
        if (d != 1) { return d; }
    }
    if (i == k) { y = x, k *= 2; }
}

vector<int64_t> factoriza(int64_t n) { // suposición: n >= 2
    vector<int64_t> factores;
    if (primalidad_miller_rabin(n)) {
        factores.push_back(n);
    } else {
        for (int64_t i = 2; ; i++) {
            auto checar = factor_pollard_rho(n, i);
            if (checar != n) {
                vector<int64_t> factores1 = factoriza(checar);
                vector<int64_t> factores2 = factoriza(n / checar);
                factores.insert(factores.end( ), factores1.begin( ),
                    ↪ factores1.end( ));
                factores.insert(factores.end( ), factores2.begin( ),
                    ↪ factores2.end( ));
                break;
            }
        }
    }
    return factores;
}
```

Inverso modular

Descripción: Encuentra <i>x</i> en $[0, m)$ tal que $ax \equiv 1 \pmod m$.	
Complejidad: $\mathcal{O}(\log(m))$	b0d623, 3 líneas

```
constexpr int64_t inverso(int64_t a, int64_t m) {
    return a <= 1 ? a : m - (m/a) * inverso(m % a, m) % m;
}
```

Matriz como clase

Descripción: Operaciones básicas con matrices.	
Uso: matriz<2, 2>({{ { 0, 1 }, { 1, 1 } }})	
Complejidad: $\mathcal{O}(n^3)$	4dc7ac, 29 líneas

```
template<class T, int F, int C>
struct matriz : array<array<T, C>, F> {
    explicit matriz(bool identidad = false) {
        for (int i = 0; i < F; ++i) {
```

```
        for (int j = 0; j < C; ++j) {
            (*this)[i][j] = (i == j && identidad);
        }
    }
}
explicit matriz(const array<array<T, C>, F>& m)
: array<array<T, C>, F>(m) {}

template<int D>
matriz<T, F, D> operator*(const matriz<T, C, D>& m) {
    matriz<T, F, D> res;
    for (int i = 0; i < F; ++i) {
        for (int j = 0; j < D; ++j) {
            for (int k = 0; k < C; ++k) {
                res[i][j] += (*this)[i][k] * m[k][j];
            }
        }
    }
    return res;
}
void operator*=(const matriz<T, F, C>& m) {
    *this = *this * m;
}
};
```

Potencia rápida generalizada

Descripción: Calcula a^b para cualquier a que pertenezca a un monoide.

Complejidad: $\mathcal{O}(\log b)$. b2ed1d, 10 líneas

```
template<typename T>
T potencia(T a, int b) {
    T res(1);
    for (; b != 0; b /= 2, a *= a) {
        if (b % 2 == 1) {
            res *= a;
        }
    }
    return res;
}
```

Potencia rápida modular

Descripción: Calcula $a^b \bmod c$.

Complejidad: $\mathcal{O}(\log b)$ 402fbc, 11 líneas

```
int64_t potencia(int64_t b, int64_t e, int64_t mod) {
    int64_t res = 1; b %= mod;
    while (e != 0) {
        if (e % 2 == 1) {
            res = (__int128_t(res) * b) % mod;
        }
        b = (__int128_t(b) * b) % mod;
        e /= 2;
    }
    return res;
}
```

Primalidad Miller Rabin

Descripción: Prueba de primalidad determinista de Miller-Rabin.

Complejidad: $\mathcal{O}(\log p)$.

Requiere: Potencia rápida modular fff63e, 32 líneas

```
bool es_primo(int64_t n) {
    if (n < 2) {
        return false;
    }

    int64_t d = n - 1, s = 0;
    while (d % 2 == 0) {
        d /= 2, s += 1;
    }
    auto compuesto_con = [&](int64_t a) {
        int64_t x = potencia(a, d, n);
        if (x == 1 || x == n - 1) {
            return false;
        }
        for (int64_t r = 1; r < s; ++r) {
            x = (__int128_t(x) * x) % n;
            if (x == n - 1) {
                return false;
            }
        }
        return true;
    };

    for (int64_t a : { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37 }) {
```

```
        if (n == a) {
            return true;
        } else if (compuesto_con(a)) {
            return false;
        }
    }
    return true;
}
```

Búsqueda y ordenamiento (3)

Búsqueda binaria generalizada

Descripción: Devuelve el primer x tal que $pred(x)$ es verdadero, y para todo x , $pred(x)$ debe ser una función monótona.

Uso: `auto elemento = busqueda_binaria(0, 1000, [](auto valor) { return valor / 500 != 0; });`

Complejidad: $\mathcal{O}(\log(n))$ e5a056, 13 líneas

```
template<typename T, typename F>
T busqueda_binaria(T ini, T fin, F pred) {
    auto res = fin;
    while (ini != fin) {
        auto mitad = ini + (fin - ini) / 2;
        if (pred(mitad)) {
            res = mitad, fin = mitad;
        } else {
            ini = mitad + 1;
        }
    }
    return res;
}
```

Salto exponencial

Descripción: Similar a la búsqueda binaria, pero ofrece un mejor rendimiento asintótico cuando el elemento buscado está cerca del inicio de la lista.

Complejidad: $\mathcal{O}(\log(i))$, donde i es la posición del elemento buscado o donde debería estar.

Requiere: Búsqueda binaria generalizada e9b66e, 9 líneas

```
template<typename T, typename F>
T salto_exponencial(T ini, T fin, F pred) {
    auto probar = ini;
    while (probar != fin && !pred(probar)) {
        probar += min(fin - probar, probar - ini + 1);
    }

    return (probar == fin ? fin : busqueda_binaria(probar - (probar -
↪ ini) / 2, probar, pred));
}
```

Cadenas (4)

Algoritmo Knuth-Morris-Pratt

Descripción: La función `tabla_kmp[i]` calcula la longitud del prefijo más largo de la cadena s que termina en i , excluyendo la subcadena $s[0..i]$ en sí misma (por ejemplo, para $abacaba \rightarrow 0010123$). Puede usarse para encontrar todas las ocurrencias de una cadena.

Uso: `auto tabla = tabla_kmp(patron); auto ocurrencias = busca(patron, texto, tabla);`

Complejidad: $\mathcal{O}(n)$ 536d5b, 24 líneas

```
vector<size_t> tabla_kmp(const string& s) {
    vector<size_t> b = { size_t(-1) };
    for (size_t i = 0, j = -1; i < s.size( ); ++i) {
        while (j != -1 && s[i] != s[j]) {
            j = b[j];
        }
        b.push_back(++j);
    }
    return b;
}
```

```
vector<size_t> busca(const string& s, const string& t, const
↪ vector<size_t>& b) {
    vector<size_t> res;
    for (size_t i = 0, j = 0; i < t.size( ); ++i) {
        while (j != -1 && t[i] != s[j]) {
            j = b[j];
        }
        if (++j == s.size( )) {
            res.push_back(i + 1 - s.size( ));
            j = b[j];
        }
    }
```

```

    }
    return res;
}

```

Arreglo de sufijos

Descripción: Construye el arreglo de sufijos para una cadena. *suffix[i]* es el iterador del inicio del sufijo que ocupa la posición *i* en el arreglo de sufijos ordenado. La función *longest_prefix* calcula los prefijos comunes más largos para las cadenas vecinas en el arreglo de sufijos: $lcp[i] = lcp(suffix[i], suffix[i+1])$, y $lcp[n-1] = 0$. La función *substring_search* devuelve una pareja de iteradores sobre *suffix* que denotan el inicio y el fin de todos los sufijos donde la subcadena es prefijo.

Uso: suffix array sa(s.begin(), s.end());
auto [ini, fin] = sa.substring_search(b.begin(), b.end());

Complejidad: $\mathcal{O}(n \log^2(n))$ donde *n* es la longitud de la cadena, y $\mathcal{O}(k \log(n))$ para *substring_search* donde *k* es la longitud del patrón. 7b6de5, 59 líneas

```

template<typename RI>
struct suffix_array {
    RI si, sf;
    vector<int> rank;
    vector<RI> suffix;

    suffix_array(RI ri, RI rf)
    : si(ri), sf(rf), rank(si, sf), suffix(sf - si) {
        vector<int> indices(sf - si);
        iota(indices.begin( ), indices.end( ), 0);
        for (int t = 1; t <= sf - si; t *= 2) { // importante que se
            ↪ haga para sf - si == 1 pues se debe normalizar el rank
            ↪ inicial
            auto pred = [&, rank = this->rank](int i1, int i2) {
                return make_pair(rank[i1], (i1 + t < sf - si ? rank[i1 + t]
                    ↪ : -1)) < make_pair(rank[i2], (i2 + t < sf - si ? rank[i2]
                    ↪ + t] : -1));
            };
            sort(indices.begin( ), indices.end( ), cref(pred));
            for (int i = 0, r = 0; i < indices.size( ); ++i) {
                rank[indices[i]] = r;
                r += (i + 1 != indices.size( ) && pred(indices[i], indices[i]
                    ↪ + 1));
            }

            for (int i = 0; i < suffix.size( ); ++i) {
                suffix[rank[i]] = si + i;
            }
        }

        vector<int> longest_prefix() {
            vector<int> res(sf - si);
            for (int i = 0, t = 0; i < rank.size( ); ++i) {
                if (rank[i] + 1 != sf - si) {
                    t += mismatch(si + i + t, sf, suffix[rank[i] + 1] + t,
                        ↪ sf).first - (si + i + t);
                    res[rank[i]] = t;
                    t -= (t > 0);
                } else {
                    t = 0;
                }
            }
            return res;
        }

        auto substring_search(auto bi, auto bf) {
            struct comparador {
                const int pos;
                bool operator()(RI iter, char c) {
                    return iter[pos] < c;
                }
                bool operator()(char c, RI iter) {
                    return c < iter[pos];
                }
            };

            auto xi = suffix.begin( ), xf = suffix.end( );
            for (int i = 0; i < bf - bi; ++i) {
                auto temp = equal_range(xi, xf, bi[i], comparador{i});
                xi = temp.first, xf = temp.second;
            }
            return pair(xi, xf);
        }
    };
};

```

LCS con memoria lineal

Descripción: Regresa la longitud de la subsecuencia común más larga usando memoria lineal.

Complejidad: $\mathcal{O}(nm)$ fae303, 16 líneas

```

int lcs(const string& a, const string& b) {
    int mem[2][b.size( ) + 1];
    int *actual = mem[0], *previo = mem[1];

```

```

    for (int i = a.size( ); i >= 0; --i, swap(actual, previo)) {
        for (int j = b.size( ); j >= 0; --j) {
            if (i == a.size( ) || j == b.size( )) {
                actual[j] = 0;
            } else if (a[i] == b[j]) {
                actual[j] = 1 + previo[j + 1];
            } else {
                actual[j] = max(actual[j + 1], previo[j]);
            }
        }
    }
    return previo[0];
}

```

LCS con programación dinámica

Descripción: Devuelve un vector de pares de enteros (*i*, *j*) que indican, respectivamente, las posiciones de las coincidencias en *a* y *b*.

Complejidad: $\mathcal{O}(n \cdot m)$ 34cdec, 27 líneas

```

auto lcs(const string& a, const string& b) {
    int mem[a.size( ) + 1][b.size( ) + 1];
    for (int i = a.size( ); i >= 0; --i) {
        for (int j = b.size( ); j >= 0; --j) {
            if (i == a.size( ) || j == b.size( )) {
                mem[i][j] = 0;
            } else if (a[i] == b[j]) {
                mem[i][j] = 1 + mem[i + 1][j + 1];
            } else {
                mem[i][j] = max(mem[i][j + 1], mem[i + 1][j]);
            }
        }
    }

    vector<pair<int, int>> res;
    int i = 0, j = 0;
    while (i < a.size( ) && j < b.size( )) {
        if (a[i] == b[j]) {
            res.emplace_back(i++, j++);
        } else if (mem[i][j] == mem[i][j + 1]) {
            ++j;
        } else {
            ++i;
        }
    }
    return res;
}

```

Subsecuencia creciente más larga

Descripción: La función *lis_size* devuelve el tamaño de la subsecuencia creciente más larga, mientras que la función *lis* devuelve un vector de iteradores que componen dicha subsecuencia.

Uso: vector<int> arr = { 10, 22, 9, 33, 21, 50, 41, 60 };
auto tam = lis_size(arr.begin(), arr.end());

Complejidad: $\mathcal{O}(n \log n)$ 1532e5, 41 líneas

```

template<typename FI>
size_t lis_size(FI ini, FI fin) {
    vector<typename iterator_traits<FI>::value_type> valores;
    for (auto i = ini; i != fin; ++i) {
        auto cambiar = upper_bound(valores.begin( ), valores.end( ),
            ↪ *i); // upper_bound para creciente no estricta,
            ↪ lower_bound para creciente estricta
        if (cambiar == valores.end( )) {
            valores.push_back(*i);
        } else {
            *cambiar = *i;
        }
    }

    return valores.size( );
}

template<typename BI>
vector<BI> lis(BI ini, BI fin) {
    vector<BI> posiciones(1, atras);
    auto pred = [&](BI i, BI j) {
        return *i < *j;
    };

```

```

    for (auto i = ini; i != fin; ++i) {
        auto cambiar = upper_bound(posiciones.begin( ) + 1,
            ↪ posiciones.end( ), i, pred); // lower_bound para creciente
            ↪ estricta
        if (cambiar == posiciones.end( )) {
            atras.push_back(posiciones.back( ));
            posiciones.push_back(i);
        }
    }

```

```
    } else if (pred(i, *cambiar))
    ↪ { // tautología con
    ↪ upper_bound (creciente no estricta) pero no con lower_bound
    ↪ (creciente estricta)
    ↪ atras.push_back(*(cambiar - 1));
    ↪ *cambiar = i;
    } else {
    ↪ atras.emplace_back( );
    }
}

for (auto i = posiciones.end( ); i != posiciones.begin( ) + 1;
    ↪ --i) {
    ↪ *(i - 2) = atras[(i - 1) - ini];
}

return vector<BI>(posiciones.begin( ) + 1, posiciones.end( ));
}
```

Trie

Descripción: Árbol enraizado que mantiene un conjunto de cadenas. La función *inserta* añade una cadena al trie, *posicion* devuelve el nodo donde termina una cadena (o *nullptr* si no existe), *busca* verifica si una cadena completa está presente, y *prefijo* comprueba si un prefijo dado existe en el trie.

Complejidad: $\mathcal{O}(n)$

```
class trie {
public:
    bool inserta(const string& s) {
        auto actual = this;
        for (int i = 0; i < s.size( ); ++i) {
            auto& siguiente = actual->nivel_[s[i]];
            if (siguiente == nullptr) {
                siguiente = new trie;
            }
            actual = siguiente;
        }
        return actual->nivel_.emplace('\0', nullptr).second;
    }

    trie* posicion(const string& s) {
        auto actual = this;
        for (int i = 0; i < s.size( ); ++i) {
            auto iter = actual->nivel_.find(s[i]);
            if (iter == actual->nivel_.end( )) {
                return nullptr;
            }
            actual = iter->second;
        }
        return actual;
    }

    bool busca(const string& s) {
        auto pos = posicion(s);
        return pos != nullptr && pos->nivel_.find('\0') !=
            ↪ pos->nivel_.end( );
    }

    bool prefijo(const string& s) {
        auto pos = posicion(s);
        return pos != nullptr;
    }

private:
    map<char, trie*> nivel_;
};
```

Combinatorios (5)

Cadenas binarias usando recursión

Descripción: Generación de cadenas binarias usando recursión.

Complejidad: $\mathcal{O}(2^n \cdot n)$

```
int n;
bool arr[MAX];

// llamada inicial: cadenas_binarias(0)
void cadenas_binarias(int i) {
    if (i == n) {
        for (int i = 0; i < n; ++i) {
            cout << arr[i];
        }
        cout << "\n";
    } else {
        arr[i] = false;
        cadenas_binarias(i + 1);
        arr[i] = true;
        cadenas_binarias(i + 1);
    }
}
```

```
}

Cadenas numéricas usando recursión

Descripción: Generación de cadenas numéricas usando recursión.

Complejidad:  $\mathcal{O}(10^n \cdot n)$ 

e6c454, 17 líneas

int n;
int arr[MAX];

// llamada inicial: cadenas_numericas(0)
void cadenas_numericas(int i) {
    if (i == n) {
        for (int i = 0; i < n; ++i) {
            cout << arr[i] << " ";
        }
        cout << "\n";
    } else {
        for (int d = 0; d <= 9; ++d) {
            arr[i] = d;
            cadenas_numericas(i + 1);
        }
    }
}
```

Coefficiente binomial iterativo

Descripción: El número de subconjuntos de k elementos de un conjunto de n elementos, $\binom{n}{k} = \frac{n!}{k!(n-k)!}$.

Complejidad: $\mathcal{O}(k)$

```
uint64_t binomial(int n, int k) {
    uint128_t res = 1;
    // uint64_t es más rápido pero falla en casos raros
    for (int i = 1; i <= k; ++i) {
        res *= n + 1 - i;
        res /= i;
    }
    return res;
}
```

Coefficiente binomial modular

Descripción: Calcula $\binom{n}{k} \bmod m$.

Complejidad: $\mathcal{O}(\log(m))$

Requiere: Inverso modular

e61de7, 9 líneas

```
int64_t factorial[N + 1]; // N dado por el problema

int64_t binomial(int n, int k, int64_t m) {
    return factorial[n] * inverso(factorial[k] * factorial[n - k] % m,
    ↪ m) % m;
}
```

Código Lehmer

Descripción: Permutaciones a/desde enteros. La biyección preserva el orden lexicográfico.

Complejidad: $\mathcal{O}(n^2)$

```
constexpr size_t factorial(size_t n) {
    return (n == 0 ? 1 : n * factorial(n - 1));
}

template<typename T>
size_t indice(T ini, T fin) {
    size_t n = fin - ini, r = factorial(n - 1), res = 0;
    for (T i = ini; i != fin; ++i) {
        res += r * count_if(i + 1, fin, [&](size_t v) { return v < *i;
        ↪ });
        if (fin - i - 1 != 0) {
            r /= fin - i - 1;
        }
    }

    return res;
}

template<typename T>
void permutacion(T ini, T fin, size_t indice) {
    iota(ini, fin, size_t(0));
    size_t n = fin - ini, r = factorial(n - 1);
    for (T i = ini; i != fin; ++i) {
        size_t d = indice / r; indice %= r;
        rotate(i, i + d, i + d + 1);
        if (fin - i - 1 != 0) {
            r /= fin - i - 1;
        }
    }
}
```

```
    }
  }
}
```

Permutaciones usando la biblioteca

Descripción: Generación de permutaciones usando la biblioteca.

Complejidad: $\mathcal{O}(n! \cdot n)$ ea4a9f, 5 líneas

```
int n;
int arr[MAX]; // inicializar con { 0, 1, ..., n - 1 }
do {
    // procesar
} while (next_permutation(&arr[0], &arr[n]));
```

Permutaciones usando recursión

Descripción: Generación de permutaciones usando recursión.

Complejidad: $\mathcal{O}(n! \cdot n)$ ee428a, 18 líneas

```
int n;
int arr[MAX]; // inicializar con { 0, 1, ..., n - 1 }

// llamada inicial: permutaciones(0)
void permutaciones(int i) {
    if (i == n) {
        for (int i = 0; i < n; ++i) {
            cout << arr[i] << " ";
        }
        cout << "\n";
    } else {
        for (int j = i; j < n; ++j) {
            swap(arr[i], arr[j]);
            permutaciones(i + 1);
            swap(arr[i], arr[j]);
        }
    }
}
```

Estructuras de datos (6)

Árbol de Fenwick

Descripción: Calcula las sumas parciales $a[i] + a[i + 1] + \dots + a[f - 1]$, y actualiza o reemplaza elementos individuales $a[i]$.

Complejidad: $\mathcal{O}(\log n)$ 9baa26, 51 líneas

```
template<typename T>
class fenwick_tree {
public:
    fenwick_tree(int n)
    : mem_(n + 1) {
    }

    int size( ) const {
        return mem_.size( ) - 1;
    }

    T operator[](int i) const {
        return query(i, i + 1);
    }

    T query(int i, int f) const {
        return query_until(f) - query_until(i);
    }

    T query_until(int f) const {
        T res = 0;
        for (; f != 0; f -= (f & -f)) {
            res += mem_[f];
        }
        return res;
    }

    int min_prefix(T v) const { // calcula la cantidad mínima
        ↪ de elementos (comenzando por la izquierda) que se necesitan
        ↪ para lograr un acumulado >= v;
        int i = 0; // si es imposible lograr dicha
        ↪ suma, regresa .size( ) + 1
        for (int j = bit_floor(mem_.size( )); j > 0; j /= 2) {
            if (i + j < mem_.size( ) && mem_[i + j] < v) {
                v -= mem_[i + j];
                i += j;
            }
        }
        return i + (v > 0);
    }

    void replace(int i, const T& v) {
```

```
        modify_add(i, v - operator[](i));
    }

    void modify_add(int i, const T& d) {
        for (i += 1; i < mem_.size( ); i += (i & -i)) {
            mem_[i] += d;
        }
    }

private:
    vector<T> mem_;
};
```

Árbol de Fenwick multidimensional

Descripción: Realiza actualizaciones y consultas de rango en múltiples dimensiones.

Uso: fenwick_tree<int, 2> arbol_2d(3, 3);
int suma = arbol_2d.query(1, 1, 3, 3); // $\sum_{i=1}^2 \sum_{j=1}^2 a[i][j]$

Complejidad: $\mathcal{O}\left(4^d \log n_1 \cdot \log n_2 \cdots \log n_d\right)$. Es eficiente para dimensiones bajas (hasta 4 o 5). eb59c9, 67 líneas

```
template<typename T, int D>
class fenwick_tree {
public:
    template<typename... P>
    fenwick_tree(int n, const P&... s)
    : mem_(n + 1, fenwick_tree<T, D - 1>(s...)) {
    }

    template<typename... P>
    void modify_add(int i, const P&... s) {
        for (i += 1; i < mem_.size( ); i += (i & -i)) {
            mem_[i].modify_add(s...);
        }
    }

    /*template<typename... P>
    T operator[](const P&... x) { // C++23
        return query(x..., (x + 1)...);
    }*/

    template<typename... P>
    T operator()(const P&... x) { // C++20 o menor
        return query(x..., (x + 1)...);
    }

    template<typename... P>
    T query(const P&... x) {
        static_assert(sizeof...(P) == 2 * D);
        return query(make_index_sequence<D>( ), array<int, 2 * D>{ x...
            ↪ });
    }

    template<typename... P>
    T query_until(int f, const P&... s) const {
        T res = 0;
        for (; f != 0; f -= (f & -f)) {
            res += mem_[f].query_until(s...);
        }
        return res;
    }

private:
    template<size_t... I, typename... P>
    T query(index_sequence<I...> i, const array<int, 2 * D>& indices)
    ↪ {
        T res = 0;
        for (unsigned i = 0; i < (1 << D); ++i) {
            res += (popcount(i) % 2 == D % 2 ? +1 : -1) *
                ↪ query_until(indices[bool(i & (1 << I)) * D + I]...);
        }
        return res;
    }

    vector<fenwick_tree<T, D - 1>> mem_;
};

template<typename T>
class fenwick_tree<T, 0> {
public:
    void modify_add(const T& d) {
        v_ += d;
    }

    T query_until( ) const {
        return v_;
    }

private:
    T v_ = 0;
};
```

Árbol de segmentos

Descripción: Estructura de datos para monóides $(T, \cdot : T \times T \rightarrow T, n \in T)$, permite realizar actualizaciones de elementos y calcular el producto de los elementos en un intervalo.

Uso: `auto s = segment_tree(move(vector_inicial), 0, plus( ));`
`int suma1 = s.query(5, 10);`
`s.replace(i, rand( ));`
`int suma2 = s.query(5, 10);`

Complejidad: $\mathcal{O}(\log n)$, se asume que f es de tiempo constante. 3b8948, 63 líneas

```
template<typename T, typename F = const T&(*)(const T&, const T&)>
class segment_tree {
public:
    segment_tree(vector<T>&& init, T v0, F f)
    : neutro(move(v0)), funcion(move(f)) {
        pisos.push_back(move(init));
        while (pisos.back( ).size( ) > 1) {
            pisos.emplace_back(pisos.back( ).size( ) / 2);
            for (int i = 0, t = pisos.size( ) - 2; i < pisos[t].size( ) /
                ↳ 2; ++i) {
                pisos.back( )[i] = funcion(pisos[t][2 * i], pisos[t][2 * i +
                    ↳ 1]);
            }
        }
    }

    int size( ) const {
        return pisos[0].size( );
    }

    const T& operator[](int i) const {
        return pisos[0][i];
    }

    void replace(int i, T v) {
        for (int p = 0;; ++p, i /= 2) {
            pisos[p][i] = move(v);
            if (i + (i % 2 == 0) == pisos[p].size( )) {
                break;
            }
            v = funcion(pisos[p][i - i % 2], pisos[p][i - i % 2 + 1]);
        }
    }

    T query(int ini, int fin) const {
        T res = neutro;
        visit(ini, fin, [&](const T& valor) {
            res = funcion(res, valor);
        });
        return res;
    }

    template<typename V>
    void visit(int ini, int fin, V&& vis) const { // callback sobre
        ↳ los nodos más representativos dentro de un rango específico
        for (int p = 0; ini != fin; ++p, ini /= 2, fin /= 2) {
            if (ini % 2 == 1) {
                vis(pisos[p][ini++]);
            }
            if (fin % 2 == 1) {
                vis(pisos[p][--fin]);
            }
        }
    }

private:
    vector<vector<T>> pisos;
    T neutro;
    F funcion;
};
```

```
// < C++17
/*template<typename T, typename F = const T&(*)(const T&, const T&)>
auto make_segment_tree(vector<T> inicial, T v0, F f) {
    return segment_tree<T, F>(move(inicial), move(v0), move(f));
}*/
```

Árbol de segmentos perezoso

Descripción: Árbol de segmentos con capacidad para modificar valores de intervalos grandes y calcular consultas de intervalos.

Uso: `auto s = lazy_segment_tree(move(vector_inicial), 0, 0, plus( ),`

```
    [](int& valor, int cubiertos, int cambio) {
        valor += cambio * cubiertos;
        return true;
    },
    [](int cambio1, int cambio2) {
        return cambio1 + cambio2;
    });
    int suma1 = s.query(5, 10);
    s.arbol.update_with(2, 8, +1);
    int suma2 = s.query(5, 10);
```

Complejidad: $\mathcal{O}(\log n)$

9050c5, 77 líneas

```
template<typename T, typename U, typename FQ = const T&(*)(const T&,
    ↳ const T&), typename FU = bool&(*)(T&, int, const U&), typename FP
    ↳ = const U&(*)(const U&, const U&)>
class lazy_segment_tree {
public:
    lazy_segment_tree(vector<T>&& init, T v0, U u0, FQ fq, FU fu, FP
        ↳ fp)
    : mem(init.size( ) * 2), neutro(move(v0)),
        ↳ neutro_update(move(u0)), funcion(move(fq)),
        ↳ funcion_update(move(fu)), funcion_propagar(move(fp)) {
        auto p = init.data( );
        construye(0, 0, size( ), p);
    }

    int size( ) const {
        return mem.size( ) / 2;
    }

    T operator[](int i) const {
        return query(i, i + 1);
    }

    T query(int ini, int fin) const {
        T res = neutro;
        visit(0, ini, fin, 0, size( ), [&](const pair<T, U>& actual, int
            ↳ cubiertos) {
            res = funcion(res, actual.first);
        });
        return res;
    }

    void update_with(int ini, int fin, const U& v) {
        visit(0, ini, fin, 0, size( ), [&](pair<T, U>& actual, int
            ↳ cubiertos) {
            actualiza(actual, cubiertos, v);
        });
    }

    template<typename V>
    void visit(int ini, int fin, V&& vis) const {
        visit(0, ini, fin, 0, size( ), [&](const pair<T, U>& actual, int
            ↳ cubiertos) {
            vis(actual.first, cubiertos);
        });
    }

private:
    const pair<T, U>& construye(int i, int ini, int fin, T& p) {
        if (fin - ini == 1) {
            return mem[i] = { move(*p++), neutro_update };
        } else {
            int tam = fin - ini, mitad = ini + tam / 2, izq = i + 1, der =
                ↳ i + 2 * (tam / 2);
            return mem[i] = { funcion(construye(izq, ini, mitad, p).first,
                ↳ construye(der, mitad, fin, p).first), neutro_update };
        }
    }

    template<typename V>
    void visit(int i, int qi, int qf, int ini, int fin, V&& vis) const
        ↳ {
        if (qi == ini && qf == fin) {
            vis(mem[i], fin - ini);
        } else if (qi < qf) {
            int tam = fin - ini, mitad = ini + tam / 2, izq = i + 1, der =
                ↳ i + 2 * (tam / 2);
            actualiza(mem[izq], tam / 2, mem[i].second);
            actualiza(mem[der], tam - tam / 2, mem[i].second);
            visit(izq, qi, min(qf, mitad), ini, mitad, vis);
            visit(der, max(qi, mitad), qf, mitad, fin, vis);
            mem[i] = { funcion(mem[izq].first, mem[der].first),
                ↳ neutro_update };
        }
    }

    void actualiza(pair<T, U>& actual, int cubiertos, const U& cambio)
        ↳ const {
```



```

    if (cambio != neutro_update && funcion_update(actual.first,
        ↪ cubiertos, cambio)) {
        actual.second = (actual.second != neutro_update ?
            ↪ funcion_propagar(actual.second, cambio) : cambio);
    }
}

mutable vector<pair<T, U>> mem;
T neutro;
U neutro_update;
FQ función;
FU funcion_update;
FP funcion_propagar;
};

// < C++17 checar segment_tree

```

Árbol de segmentos persistente

Descripción: Árbol de segmentos que permite consultar cualquier versión anterior del árbol después de cada modificación.

Complejidad: $\mathcal{O}(\log n)$

693f0e, 92 líneas

```

template<typename T, typename F = const T&(*)>(const T& actual, const T& tam) {
class persistent_segment_tree {
    struct nodo {
        T valor;
        nodo *izq, *der;
    };
}

```

```

public:
template<typename I>
persistent_segment_tree(T n, F f, I&& entrada, int t)
: mem_(make_shared<deque<nodo>>()), neutro_(move(n)),
  ↪ funcion_(move(f)), tam_(t), raiz_(replace(nullptr, 0, tam_, 0,
  ↪ tam_, entrada)) {
}

```

```

template<typename RI>
persistent_segment_tree(T n, F f, RI ini, RI fin)
: persistent_segment_tree(move(n), move(f), [&]( ) { return
  ↪ *ini++; }, fin - ini) {
}

```

```

int size( ) const {
    return tam_;
}

```

```

const T& operator[](int i) const {
    const T* res;
    visit(i, i + 1, [&](const T& actual) {
        res = &actual;
    });
    return *res;
}

```

```

T query(int ini, int fin) const {
    T res = neutro_;
    visit(ini, fin, [&](const T& actual) {
        res = funcion_(res, actual);
    });
    return res;
}

```

```

persistent_segment_tree replace(int i, T v) {
    return { neutro_, funcion_, tam_, replace(raiz_, i, i + 1, 0,
        ↪ tam_, [&]( ) { return move(v); }), mem_ };
}

```

```

template<typename V>
void visit(int ini, int fin, V&& vis) const {
    return visit(raiz_, ini, fin, 0, tam_, vis);
}

```

```

private:
persistent_segment_tree(T n, F f, int t, nodo* r,
    ↪ shared_ptr<deque<nodo>>& m)
: mem_(m), neutro_(move(n)), funcion_(move(f)), raiz_(r), tam_(t)
  ↪ {
}

```

```

template<typename I>
nodo* replace(nodo* p, int qi, int qf, int ini, int fin, I&&
  ↪ entrada) {
    if (ini == fin || qi >= qf) {
        return p;
    } else if (fin - ini == 1) {
        return crea(entrada( ));
    } else {
        int mitad = ini + (fin - ini) / 2, tam = fin - ini;
        auto izq = replace((p == nullptr ? nullptr : p->izq), qi,
            ↪ min(qf, mitad), ini, mitad, entrada);
        auto der = replace((p == nullptr ? nullptr : p->der), max(qi,
            ↪ mitad), qf, mitad, fin, entrada);
    }
}

```

```

    return crea(funcion_(izq->valor, der->valor), izq, der);
}
}

```

```

template<typename V>
void visit(const nodo* p, int qi, int qf, int ini, int fin, V&
  ↪ vis) const {
    if (qi >= qf) {
        return;
    } else if (qi == ini && qf == fin) {
        vis(p->valor);
    } else {
        int mitad = ini + (fin - ini) / 2;
        visit(p->izq, qi, min(qf, mitad), ini, mitad, vis);
        visit(p->der, max(qi, mitad), qf, mitad, fin, vis);
    }
}
}

```

```

template<typename... P>
nodo* crea(P&&... v) {
    return &mem_>insert(mem_>end( ), nodo{forward<P>(v)...});
}

```

```

shared_ptr<deque<nodo>> mem_;
T neutro_;
F funcion_;
int tam_;
nodo* raiz_;
};

```

// < C++17 checar segment_tree

Geométricos (7)

Cerco convexo

Descripción:

Devuelve un vector de los puntos del cerco convexo en sentido antihorario. Los puntos en el borde del cerco entre dos puntos no se consideran parte del cerco.



Uso: sort(puntos.begin(), puntos.end());
 auto cerco = cerco_convexo(puntos.begin(), puntos.end());

Complejidad: $\mathcal{O}(n \log n)$

d47d2d, 46 líneas

```

auto producto_cruz(const auto& a, const auto& b, const auto& c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

```

```

template<typename RI1, typename RI2>
auto cerco_parcial(RI1 ai, RI1 af, RI2 bi) {
    auto bw = bi;
    for (; ai != af; *bw++ = *ai++) {
        while (bw - bi >= 2 && producto_cruz(*(bw - 2), *(bw - 1), *ai)
            ↪ <= 0) { // < 0 para permitir empates en línea recta
            --bw;
        }
    }
    return bw;
}

```

```

template<typename RI>
auto cerco_convexo(RI ai, RI af) {
    if (af - ai <= 2) {
        return vector<typename iterator_traits<RI>::value_type>(ai, af);
    }
    vector<typename iterator_traits<RI>::value_type> res(2 * (af -
        ↪ ai));
    auto iter1 = cerco_parcial(ai, af, res.begin( )) - 1;
    auto iter2 = cerco_parcial(make_reverse_iterator(af),
        ↪ make_reverse_iterator(ai), iter1) - 1;
    res.resize(iter2 - res.begin( ));
    return res;
}

```

```

auto distancia(const auto& a, const auto& b) {
    return hypot(a.x - b.x, a.y - b.y); // hypot puede ser más lento
    ↪ que hacerlo manualmente con sqrt
}

```

```

template<typename T>
auto perimetro(const vector<T>& puntos) {
    double res = 0;
    for (int i = 0; i < puntos.size( ); ++i) {
        res += distancia(puntos[i], puntos[(i + 1) % puntos.size( )]);
    }
    return res;
}

```

```

struct punto {
    double x, y; // si no necesitan doubles, pasarlos a int porque
    ↪ es más rápido
    bool operator<(const punto& p) const {
        return pair(x, y) < pair(p.x, p.y);
    }
}

```

```
}
};
```

Distancia entre dos puntos

Descripción: Calcula la distancia euclidiana entre dos puntos en un plano.

Uso: `double dis = distancia<int>({ 0, 0 }, { 1, 1 });` 31a15d, 6 líneas

```
template<typename T>
double distancia(const pair<T, T>& a, const pair<T, T>& b) {
    auto dx = a.first - b.first;
    auto dy = a.second - b.second;
    return sqrt(dx * dx + dy * dy);
}
```

Gráficas (8)

Algoritmo de Floyd

Descripción: Calcula el camino más corto entre cualquier pareja de vértices en un grafo dirigido.

Complejidad: $\mathcal{O}(n^3)$ f7e22a, 9 líneas

```
// adyacencia[i][j] es la matriz original
// la siguiente implementación la modifica
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            adyacencia[i][j] = min(adyacencia[i][j], adyacencia[i][k] +
                                   ↪ adyacencia[k][j]);
        }
    }
}
```

Algoritmo de Hopcroft-Karp

Descripción: Algoritmo rápido de acoplamiento bipartito. Devuelve el tamaño del acoplamiento. *match_b[j]* será el vértice del lado izquierdo que quedó emparejado con el vértice *j* del lado derecho, o *-1* si no está emparejado.

Uso: `bipartito_hopcroft bp(|A|, |B|);`
`bp.agrega_arco(u, v);` // por cada arista ($u \in A, v \in B$)
`int tam = bp.calcula_acoplamiento();`

Complejidad: $\mathcal{O}(\sqrt{nm})$ 4c8140, 64 líneas

```
struct bipartito_hopcroft {
    vector<vector<int>> adyacencia;
    vector<int> match_b;

    bipartito_hopcroft(int n, int m)
        : adyacencia(n), match_b(m, -1) {}

    void agrega_arco(int i, int j) {
        adyacencia[i].push_back(j);
    }

    int calcula_acoplamiento() {
        for (;;) {
            vector<int> capas = calcula_capas();
            bool mejora = false, visto_a[adyacencia.size()] = {};
            for (int i = 0; i < adyacencia.size(); ++i) {
                mejora |= (capas[i] == 0 && aumenta(i, visto_a, capas));
            }
            if (!mejora) {
                return match_b.size() - count(match_b.begin(),
                                                ↪ match_b.end(), -1);
            }
        }
    }

private:
    vector<int> calcula_capas() {
        vector<int> capas(adyacencia.size(), 0);
        deque<int> cola(adyacencia.size());
        iota(cola.begin(), cola.end(), 0);
        for (int j = 0; j < match_b.size(); ++j) {
            int i = match_b[j];
            if (i != -1) {
                cola[i] = capas[i] = -1;
            }
        }

        cola.erase(remove(cola.begin(), cola.end(), -1), cola.end());
        for (; !cola.empty(); cola.pop_front()) {
            int i = cola.front();
            for (int j : adyacencia[i]) {
                if (match_b[j] != -1 && capas[match_b[j]] == -1) {
                    capas[match_b[j]] = capas[i] + 1;
                    cola.push_back(match_b[j]);
                }
            }
        }
    }
}
```

```
    }
}

return capas;
}

bool aumenta(int i, bool visto_a[], const vector<int>& capas) {
    if (!visto_a[i]) {
        visto_a[i] = true;
        for (int j : adyacencia[i]) {
            if (match_b[j] == -1 || capas[i] + 1 == capas[match_b[j]] &&
                ↪ aumenta(match_b[j], visto_a, capas)) {
                match_b[j] = i;
                return true;
            }
        }
    }
    return false;
}
};
```

Algoritmo de Kruskal

Descripción: Dado un grafo no dirigido y ponderado, se busca encontrar un árbol abarcador con el menor costo posible.

Complejidad: Complejidad: $\mathcal{O}(m \log n)$

Requiere: Unión-pertenencia e6da5a, 27 líneas

```
struct arista {
    int x, y, costo;
};

int main() {
    int v, a;
    cin >> v >> a;

    vector<arista> aristas;
    for (int i = 0; i < a; ++i) {
        int x, y, costo;
        cin >> x >> y;
        aristas.push_back({ x, y, costo });
    }

    sort(aristas.begin(), aristas.end(), [](arista a, arista b) {
        return a.costo < b.costo;
    });

    union_find uf(v);

    for (int i = 0; i < aristas.size(); ++i) {
        if (!uf.connected(aristas[i].x, aristas[i].y)) {
            uf.join(aristas[i].x, aristas[i].y);
        }
    }
}
```

Algoritmo de Kuhn

Descripción: Algoritmo simple de acoplamiento bipartito. Devuelve el tamaño del acoplamiento. *match_b[j]* será el vértice del lado izquierdo que quedó emparejado con el vértice *j* del lado derecho, o *-1* si no está emparejado.

Uso: `bipartito_kuhn bp(|A|, |B|);`
`bp.agrega_arco(u, v);` // por cada arista ($u \in A, v \in B$)
`int tam = bp.calcula_acoplamiento();`

Complejidad: $\mathcal{O}(nm)$ 21987f, 35 líneas

```
struct bipartito_kuhn {
    vector<vector<int>> adyacencia;
    vector<int> match_b;

    bipartito_kuhn(int n, int m)
        : adyacencia(n), match_b(m, -1) {}

    void agrega_arco(int i, int j) {
        adyacencia[i].push_back(j);
    }

    int calcula_acoplamiento() { // sólo llamar una vez
        int res = 0;
        for (int i = 0; i < adyacencia.size(); ++i) {
            bool visto_a[adyacencia.size()] = {};
            res += aumenta(i, visto_a);
        }
        return res;
    }

private:
    bool aumenta(int i, bool visto_a[]) {
        if (!visto_a[i]) {

```

```

    visto a[i] = true;
    for (int j : adyacencia[i]) {
        if (match b[j] == -1 || aumenta(match_b[j], visto_a)) {
            match_b[j] = i;
            return true;
        }
    }
    return false;
}
};

```

Algoritmos de Dijkstra y Prim

Descripción: El algoritmo de Dijkstra encuentra los caminos más cortos desde un vértice inicial s hacia todos los demás vértices del grafo con costos no negativos. El árbol abarcador de costo mínimo, que se obtiene usando el algoritmo de Prim, es un conjunto de aristas que conecta todos los vértices con el menor costo total posible.

Complejidad: $\mathcal{O}(m \log n)$

32131c, 41 líneas

```

struct tupla {
    int origen, destino, costo;
};

bool operator<(tupla a, tupla b) {
    return a.costo > b.costo;
}

int main( ) {
    int n, m;
    cin >> n >> m;

    vector<tupla> adyacencia[n];
    for (int i = 0; i < m; ++i) {
        int x, y, c;
        cin >> x >> y >> c;
        adyacencia[x].push_back({ x, y, c });
        adyacencia[y].push_back({ y, x, c });
    }

    priority_queue<tupla> cp;
    cp.push({0, 0, 0});
    int costo[n], previo[n];
    fill(&costo[0], &costo[n], -1);
    do {
        tupla t = cp.top( );
        cp.pop( );
        if (costo[t.destino] == -1) {
            costo[t.destino] = t.costo;
            previo[t.destino] = t.origen;
            for (tupla vecino : adyacencia[t.destino]) {
                vecino.costo += t.costo; // quitar para Prim
                cp.push(vecino);
            }
        }
    } while (!cp.empty( ));

    for (int i = 0; i < n; ++i) {
        cout << i << ": " << costo[i] << " " << previo[i] << "\n";
    }
}

```

Circuito euleriano

Descripción: Un circuito euleriano es un camino que recorre cada arista de un grafo exactamente una vez, y el camino termina en el vértice *inicial*.

Complejidad: $\mathcal{O}(n + m)$

c6a5d6, 40 líneas

```

struct punto {
    double x, y;
};

struct arco {
    int x, y;
};

double factor_windy(punto p1, punto p2) {
    return (p1.x <= p2.x ? 1 / 1.5 : 2);
}

double distancia_windy(punto p1, punto p2) {
    return hypot(factor_windy(p1, p2) * (p1.x - p2.x), p1.y - p2.y);
}

struct por_costo {
    const punto* arr;
    bool operator()(const arco& a, const arco& b) const {
        return tuple(distancia_windy(arr[a.x], arr[a.y]), a.x, a.y) <
            tuple(distancia_windy(arr[b.x], arr[b.y]), b.x, b.y);
    }
};

```

```

vector<int> hierholzer(int inicial, vector<set<arco, por_costo>>&
    ↪ adyacencia) {
    vector<int> pila = { inicial }, res;
    do {
        int actual = pila.back( );
        if (!adyacencia[actual].empty( )) {
            auto [origen, destino] = *adyacencia[actual].begin( );
            adyacencia[actual].erase(adyacencia[actual].begin( ));
            adyacencia[destino].erase(arco(destino, actual));
            pila.push_back(destino);
        } else {
            res.push_back(actual);
            pila.pop_back( );
        }
    } while (!pila.empty( ));
    reverse(res.begin( ), res.end( ));
    return res;
}

```

Flujo máximo con costo mínimo

Descripción: .

Uso: ford_fulkerson $f(n, s, t)$;
 $f.agrega_arco(u, v, c, d)$;
 $auto [flujo, costo] = f.flujo_maximo()$;

Complejidad: $\mathcal{O}()$

fbf404, 65 líneas

```

struct ford_fulkerson {
    int vertices, fuente, sumidero;
    vector<vector<int>> capacidad;
    vector<vector<int>> costo;
    vector<vector<int>> vecinos;

    ford_fulkerson(int v, int s, int t)
    : vertices(v), fuente(s), sumidero(t), capacidad(v,
    ↪ vector<int>(v)), costo(v, vector<int>(v)), vecinos(v) {
    }

    void agrega_arco(int i, int j, int c, int d) {
        capacidad[i][j] = c;
        costo[i][j] = d;
        costo[j][i] = -d;
        vecinos[i].push_back(j);
        vecinos[j].push_back(i);
    }

    pair<int, vector<pair<int, int>>> camino_aumentante( ) {
        vector<int> anterior(vertices, -1);
        vector<int> costo_minimo(vertices, 1e9);
        costo_minimo[fuente] = 0;

        for (int k = 0; k < vertices - 1; ++k) {
            for (int i = 0; i < vertices; ++i) {
                for (int j : vecinos[i]) {
                    if (capacidad[i][j] > 0 && costo_minimo[j] >
                    ↪ costo_minimo[i] + costo[i][j]) {
                        costo_minimo[j] = costo_minimo[i] + costo[i][j];
                        anterior[j] = i;
                    }
                }
            }
        }

        if (costo_minimo[sumidero] == 1e9) {
            return { 0, { } };
        }

        vector<pair<int, int>> camino;
        int cuello = 1e9, actual = sumidero;
        do {
            cuello = min(cuello, capacidad[anterior[actual]][actual]);
            camino.emplace_back(anterior[actual], actual);
            actual = anterior[actual];
        } while (actual != fuente);

        return { cuello, camino };
    }

    pair<int, int> flujo_maximo( ) {
        int flujo_total = 0, costo_total = 0;
        for (;;) {
            auto [aumento_flujo, camino] = camino_aumentante( );
            if (aumento_flujo == 0) {
                return { flujo_total, costo_total };
            }
            flujo_total += aumento_flujo;
            for (auto [i, j] : camino) {
                costo_total += aumento_flujo * costo[i][j];
                capacidad[i][j] -= aumento_flujo;
                capacidad[j][i] += aumento_flujo;
            }
        }
    }
}

```

```
};
```

Puentes de una gráfica

Descripción: Dado un grafo no dirigido. Un puente se define como una arista que, al ser removida, desconecta el grafo (o, más precisamente, aumenta el número de componentes conexos en el grafo). Devuelve todos los puentes en el grafo dado.

Complejidad: $\mathcal{O}(n + m)$ 140d10, 31 líneas

```
void dfs(int actual, int anterior, const vector<int> adyacencia[],
↪ vector<bool>& visitado, int& id, vector<int>& tin, vector<int>&
↪ low, vector<pair<int, int>>& res) {
    visitado[actual] = true, tin[actual] = low[actual] = id++;
    bool anterior_omitido = false;
    for (int vecino : adyacencia[actual]) {
        if (vecino == anterior && !anterior_omitido) {
            anterior_omitido = true;
            continue;
        }
        if (visitado[vecino]) {
            low[actual] = min(low[actual], tin[vecino]);
        } else {
            dfs(vecino, actual, adyacencia, visitado, id, tin, low, res);
            low[actual] = min(low[actual], low[vecino]);
            if (low[vecino] > tin[actual]) {
                res.emplace_back(min(actual, vecino), max(actual, vecino));
            }
        }
    }
}

vector<pair<int, int>> calcula_puentes(const vector<int>
↪ adyacencia[], int n) {
    vector<bool> visitado(n); int id;
    vector<int> tin(n, -1), low(n, -1);
    vector<pair<int, int>> res;
    for (int i = 0; i < n; ++i) {
        if (!visitado[i]) {
            dfs(i, -1, adyacencia, visitado, id, tin, low, res);
        }
    }
    return res;
}
```

Unión-pertenencia

Descripción: Dado un grafo no dirigido, procesa la adición de aristas, verificar si dos vértices están en el mismo componente conexo y obtener el tamaño del componente de un elemento.

Complejidad: $\mathcal{O}(\alpha(n))$ (amortizado). f11bf1, 30 líneas

```
struct union_find {
    vector<int> data;

    union_find(int n)
    : data(n, -1) {}

    int leader(int i) {
        return (data[i] < 0 ? i : data[i] = leader(data[i]));
    }

    int size(int i) {
        return -data[leader(i)];
    }

    bool connected(int i, int j) {
        return leader(i) == leader(j);
    }

    void join(int i, int j) {
        int ri = leader(i), rj = leader(j);
        if (ri != rj) {
            if (-data[ri] < -data[rj]) {
                swap(ri, rj);
            }
            data[ri] += data[rj];
            data[rj] = ri;
        }
    }
};
```

Lectura y escritura (9)

Construcción de cadenas

Descripción: Ejemplo de concatenación de valores de diferentes tipos de datos en una sola cadena. 3b1190, 11 líneas

```
int main( ) {
    int a = 57;
    char c = '@';
    double f = 3.14;

    ostreamstream bufer;
    bufer << a << " " << c << " " << f;
    string cadena = bufer.str( );

    cout << cadena;
}
```

Tokenización de líneas

Descripción: Ejemplo de tokenización de líneas de texto, donde cada línea leída se divide en palabras que se imprimen individualmente. f9cb58, 11 líneas

```
int main( ) {
    string linea;
    while (getline(cin, linea)) {
        istringstream extractor(linea);
        string palabra;
        while (extractor >> palabra) {
            cout << palabra << " ";
        }
        cout << "\n";
    }
}
```